

Practical Work 3: “Document databases and hierarchical data model”

This document outlines the third practical work assignment for the course "Data models in database systems". The assignment has been made in a way to create a practical understanding of how to work with document databases and hierarchical data models using MongoDB. **This task is individual.**

You will be given a scenario description. You will be provided with necessary *.json files (e.g. airlines.json, airports.json, flight_logs.json) that contain corresponding data, and you will have to use the data to complete the tasks.

Submissions

The submission deadline is **December 28**. An emergency extension may be requested via email **before** the deadline if you have not used it before for PW1 or PW2.

Reminder: You can only use one extension per semester!

You must submit 3 items to ORTUS:

1. Report (*.pdf format):

- a. This document will contain screenshots, semantic explanations as well as technical explanations and a link to your video.
- b. There is a template “*report_template.docx*” provided in ORTUS that you **MUST** use.
- c. The submitted report file **MUST** be named like this:
 - i. “[PW3_variation_No]_[student_id]_PW3.pdf”
 - ii. If your variation is No. 7 and student ID is 191RDB001 then the files name would be 7_191RDB001_PW3.pdf

2. Commented Code file (*.js format):

- a. This file will contain **commented scripts** and **queries** that are used to complete each task.
 - i. Query: A request sent to the database to select, filter, and retrieve specific documents based on defined criteria.
 - ii. Script: A set of code instructions that creates a workflow to automate logic, perform multiple operations, or modify data in sequence.
- b. There is a template “*code_template.js*” provided in ORTUS that you **MUST** use.
- c. The file type you submit **MUST** be a *.js file.

- d. The submitted report file **MUST** be named like this:
 - e. The submitted code file **MUST** be named like this:
 - i. “[PW3_variation_No]_[student_id]_PW3.js”
 - ii. If your variation is No. 7 and student ID is 191RDB001 then the files name would be 7_191RDB001_PW3.js
3. **Presentation Video (Link in Report):** A 5–7 minute video where you explain:
- a. What is MongoDB
 - b. How you used it
 - c. Your database and it’s collections
 - d. How you imported data using MongoDB Compass
 - e. Present **1 query** from **task 5** and explain in detail each step in it

Grading and Points Distribution

The total task is worth 100 points, distributed as follows:

- **Main report (35 points total):** Submitted as [PW3_variation_No]_[student_id]_PW3.pdf (e.g. 7_191RDB001_PW3.pdf)
 - **Administrative Data:**
 - Student ID, Name, Surname, Variation Number and link to the video.
 - **Installation & Import Evidence (6 pts):**
 - **(3 pts):** Screenshots of the Installation process (task bar with date/time must be visible).
 - **(3 pts):** Screenshots of MongoDB Compass showing each of the collections after import.
 - **Written Explanations (25 pts):**
 - **Task 3 (4 pts):**
 - You must provide
 - The Semantic Explanation (Purpose)
 - Technical Explanation (Process) as defined in the task instructions.
 - Screenshots of:
 - before implementation
 - the script
 - results after implementation
 - **Task 4 (4 pts):**
 - You must provide
 - The Semantic Explanation (Purpose)
 - Technical Explanation (Process) as defined in the task instructions.
 - Screenshots of:
 - before implementation
 - the script
 - results after implementation
 - **Task 5.1 (4 pts):**
 - You must provide
 - The Semantic Explanation (Purpose)
 - Technical Explanation (Process) as defined in the task instructions.
 - Screenshots of:
 - the script
 - results of the query (if it is too long then the queries first 5 outputs)
 - **Task 5.2 (4 pts):**
 - You must provide
 - The Semantic Explanation (Purpose)

- Technical Explanation (Process) as defined in the task instructions.
 - Screenshots of:
 - the script
 - results of the query (if it is too long then the queries first 5 outputs)
- **Task 5.3 (4 pts):**
 - You must provide
 - The Semantic Explanation (Purpose)
 - Technical Explanation (Process) as defined in the task instructions.
 - Screenshots of:
 - the script
 - results of the query (if it is too long then the queries first 5 outputs)
- **Task 5.4 (4 pts):**
 - You must provide
 - The Semantic Explanation (Purpose)
 - Technical Explanation (Process) as defined in the task instructions.
 - Screenshots of:
 - the script
 - results of the query (if it is too long then the queries first 5 outputs)
- **Conclusions (5 pts):**
 - A summary of the work completed
 - **Reflection:** Discuss the challenges faced (e.g., working with data types, nesting documents) and what you learned about document databases vs. relational databases
 - Answer to theoretical question 1
 - Answer to theoretical question 2
- **Technical Implementation (35 points):** Submitted as [PW3_variation_No]_[student_id]_PW3.js (e.g. 7_191RDB001_PW3.js)
 - **Administrative Data:**
 - **(3 pts):** Student ID, Name, Surname, and **Variation Number**.
 - **Task 3: Type Correction Script:**
 - **(2 pts): First Collection:** Script must successfully convert incorrect data types (Strings) into the correct formats (Double, Int, Date, etc)
 - **(2 pts): Second Collection:** Script must successfully convert incorrect data types (Strings) into the correct formats (Double, Int, Date, etc)

- **(2 pts): Third Collection:** Script must successfully convert incorrect data types (Strings) into the correct formats (Double, Int, Date, etc)
 - **Note:** These must be update scripts that persist changes, not just read queries.
 - **Task 4: Data Integration/Denormalization:**
 - **(6 pts):** The pipeline must create a new denormalized collection.
 - **Task 5: Advanced Aggregations:**
 - **Task 5.1 (5 pts):** Query returns correct output.
 - **Task 5.2 (5 pts):** Query returns correct output.
 - **Task 5.3 (5 pts):** Query returns correct output.
 - **Task 5.4 (5 pts):** Query returns correct output.
- **Presentation Video (30 points):** Explains concepts, workflow, and detailed query analysis.
 - **Duration 5-7 minutes. Upload to cloud, link provided in Report.**
 - **Understanding MongoDB (3 pts):** Clear explanation of "What is MongoDB," specifically covering its Document-oriented nature vs Relational databases.
 - **Workflow & Usage (3 pts):** Briefly explain how you set up the environment and used the tools.
 - **Database Structure (4 pts):** Demonstration of your database and its collections as required by your variation.
 - **Import Process (10 pts):** Demonstrate and explain how you used MongoDB Compass to import the JSON files.
 - **Deep Dive Query Explanation (10 pts):** Select **one** complex query from Task 5. Explain it line-by-line, focusing on what is happening to the data.
- **Automatic failure of the work:**
 - **0 for the entire work:**
 - **Plagiarism of any kind:** If students work contains or creates significant suspicions of plagiarism (code, report or video content is copied from another student or completely or partly generated by AI) will result in an automatic failure of the entire work.
 - **Work not submitted by the final deadline:** Failure to submit the work before the deadline (with exception if you have acquired an extension) will result in an automatic failure of the entire work.
 - **You have completed the incorrect variation:** Student has submitted an incorrect variation. This includes writing in your *.pdf or *.js file wrong variation number and will result in an automatic failure of the entire work.
 - **Missing Core Files:** Failure to submit the *.js file and/or the *.pdf file will result in an automatic failure of the entire work.

- **Missing a video presentation or submitting a trivial video presentation:** Not providing a link to video presentation in report or it being trivial and lacking significant information will result in an automatic failure of the entire work.
 - **Not using the provided file templates:** Failure to use and submit files based on the provided file templates will result in an automatic failure of the entire work.
 - **Missing or Incorrect “Administrative Data”:** In the Report .pdf file and code .js file there is a specific place to write your name, student_id etc. If for any reason any part of this is missing or is incorrect it will result in an automatic failure of the entire work.
 - ***.js file is not executable:** If student submits a *.js file but the instructor is unable to run it will result in an automatic failure of the entire work.
 - **Code file type and/or name is incorrect:** The code file type is not *.js or is not named as required will result in an automatic failure of the entire work.
 - **Report file type and/or name is incorrect:** The report file type is not *.pdf or is not named as required will result in an automatic failure of the entire work.
 - **Any of the submitted files are empty:** Submitting the necessary files but any of them are empty (e.g. *.js file is submitted but does not contain any queries) will result in an automatic failure of the entire work.
 - **Code seen in screenshots does not match the one provided in *.js file:** This will be considered as an attempt of plagiarism and misleading the instructor and will result in an automatic failure of the entire work. **If there are any differences between code in screenshots and the *.js file code due to the difference between the syntax requirements between the MongoDB Compass MongoDB Shell and the .js code, then it will NOT be considered a violation.**
 - **Malicious Code:** Any attempts at damaging the database, system etc. will result in an automatic failure of the entire work.
- **Point deductions**
 - **Negative marking:** Actions that will result in a reduction to your score:
 - **(-10 pts):** Hardcoding values and/or types if the task specifically asks to do this by using code.
 - **(-5 pts):** For each screenshot that does not comply with the outlined requirements (e.g. Missing the “task bar with date/time”)
 - **(-15 pts):** If the video is under 4.5 minutes (lacks detail) or over 8 minutes.
 - **(-10 pts):** If the video presentation lacks clarity and or professionalism from the student.
 - **General task failures:** Actions that will result in not receiving points:
 - **(0 pts for a task):** Screenshot or screenshots corresponding to the task are not readable.
 - **(0 pts for a task):** A screenshot is not provided when specified in a task. This also includes:

- if the task specifically asks you to provide 5 screenshots, but you only provided 4.
 - if the task specifically asks you to provide 5 screenshots of specific parts of the task and you provide 5 screenshots, **but the images do not display the specific content requested.**
- **(0 pts for a task):** The script or query for a task is missing.
 - **(0 pts for a task):** The script or query output is incorrect.
 - **(0 pts for a task or subtask):** Missing comments or trivial comments about the script or queries in the *.pdf or *.js file.
 - It is possible to lose points for other significant errors in the code or report not mentioned in this document. These will be mentioned when you receive your grade.

Task Software

To do this task you will need to download and install **MongoDB Community Server** as well as **MongoDB Compass**.

MongoDB Community Server: <https://www.mongodb.com/try/download/community>

MongoDB Compass: <https://www.mongodb.com/try/download/compass>

Report Structure

Your report must be a single *.pdf file containing the following sections:

1. **Title Page:** Course title, work title, student ID, name, and surname and work variation number.
2. **Demonstration Video:** A clickable, accessible link to your video (e.g. YouTube unlisted or OneDrive)
3. **Task 1: Installation process:**
 - a. **A step-by-step description** of how you installed MongoDB.
 - b. **Screenshots of:**
 - i. **MongoDB Community Server** Download page with your specific download **Version, Platform** and **Package**.
 - ii. **MongoDB Compass** Download page with your specific download **Version, Platform** and **Package**.
 - iii. The "**Component Selection**" screen during installation.
 - iv. The "**Installation Complete**" screen.
 - v. A screenshot of the terminal showing a **successful connection** to mongosh.
 - vi. **Crucial Requirement:** The task bar with the **date and time** must be visible in *all* these screenshots.
4. **Task 2: Data import verification:**
 - a. **Description:** Brief explanation of creating the database and the three collections.
 - b. **Screenshots of:**
 - i. **MongoDB Compass** of the created **database**.
 - ii. **MongoDB Compass** for **each of the 3 collections**.
 - i. **Crucial Requirement:**
 1. These screenshots must clearly show that all fields in each of the collections were "String" type after importing the data.
 2. The task bar with the date and time must be visible in all of these screenshots.

5. Task 3: Data Type Correction

- a. **Process:** In detail explain the script used to fix the data types.
- b. **Screenshots of:**
 - i. **MongoDB Compass** showing that the data types have been successfully changed to their correct formats (e.g. from String to Int, Dates etc.) after running your script for each of the collections (even if a collection does not change for your specific task requirements).
 - ii. Code fragments with short comments that explain the code that was used to convert the formats from String to the necessary type.
 - iii. **Crucial Requirement:** The task bar with the date and time must be visible in all these screenshots.

6. Task 4: Denormalization:

- a. **You must provide:**
 - i. **Scripts title.**
 - ii. **Purpose (Semantic explanation):** A plain-English description of what the script is trying to achieve (one sentence).
 - iii. **Process (Technical explanation):** A step-by-step breakdown of how the code works.
- b. **Screenshots of:**
 - i. The script itself.
 - ii. The output of the script
 - iii. The new denormalized collection in MongoDB Compass
 - iv. **Crucial Requirement:** The task bar with the date and time must be visible in all these screenshots.

7. Task 5: Advanced Queries:

- a. **You must provide for each subtask:**
 - i. **Queries title.**
 - ii. **Purpose (Semantic explanation):** A plain-English description of what the script is trying to achieve (one sentence).
 - iii. **Process (Technical explanation):** A step-by-step breakdown of how the code works.
 - iv. **The result of the query.**
- b. **Screenshots of:**
 - i. The query itself.
 - ii. The output of the query.
 - iii. **Crucial Requirement:** The task bar with the date and time must be visible in all these screenshots.

8. Conclusions:

- a. **Summary:** A summary of the work completed.

- b. Reflection:** Discuss the challenges faced (e.g., working with data types, nesting documents) and what you learned about document databases vs. relational databases. **This is an extremely important part of the report.**
- c. Answer the theoretical questions**
 - i. Provide the question itself.**
 - ii. Write the answer to the question**

Commented Scripts and Queries file Structure

You have been provided with a “*code_template.js*” file. You must use it and submit your completed scripts and queries in it.

It is recommended that you use MongoDB compass inbuilt terminal while you are writing code, testing and debugging as it will be easier to see your code and check the output of separate tasks. Once you have completed all the codes you need to insert them in the “*code_template.js*” template and rename the file to “*[PW3_variation_No]_[student_id]_PW3.js*” (e.g. *7_191RDB001_PW3.js*).

IMPORTANT!

To check if your “*[PW3_variation_No]_[student_id]_PW3.js*” file works correctly run it in the computer’s terminal using mongosh.

To run the .js file:

1. Open computers terminal
2. Run “mongosh” → This should run your local database
3. Run “load("path/to/student _ id.js")”
4. You should see output of all the tasks.

Your submitted “*[PW3_variation_No]_[student_id]_PW3.js*” file must be a single .js file that follows the structure of the template.

The file **must** contain the following sections in this exact order:

1. Header information:

- a. A comment block at the very top containing:
 - i. Course Title
 - ii. Student ID, Name and Surname
 - iii. Variation number

2. Database connection command:

- a. The command to switch to the correct database (e.g. *use('aviationDB');*).
- b. **Requirement:** This ensures the instructor does not need to manually switch databases before running the script.

3. Task 3: Data Cleaning & Type Fixing:

- a. **Comments:** A brief 1-line comment explaining what fields are being fixed.
- b. **Code:** The updateMany scripts that convert Strings to Int/Double/Date etc.
- c. **Mandatory Output:** A print() statement confirming the updates were executed (e.g., *print("Task 3: Data types fixed.");*).

4. Task 4: Denormalization (Data Integration):

- a. **Comments:** Explanation of which collections are being joined.
- b. **Code:** The aggregation pipeline that creates the new collection.
- c. **Mandatory Output:** A `print()` statement confirming the new collection was created (e.g., `print("Task 4: flights_enhanced collection created.");`).

5. Task 5: Analytical Queries:

- a. **Critical Requirement:** This section must not just *define* the queries but execute and print them.
- b. **Format per Query (5.1, 5.2, 5.3, 5.4):**
 - i. **Title Comment:** (e.g., *// 5.1 Airline Performance*)
 - ii. **The Query:** The aggregation code.
 - iii. **Print Command:** The result must be wrapped in `printjson(...
 .toArray())` so the instructor sees the output immediately in the terminal.